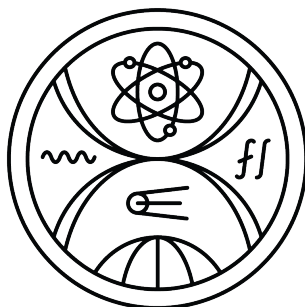


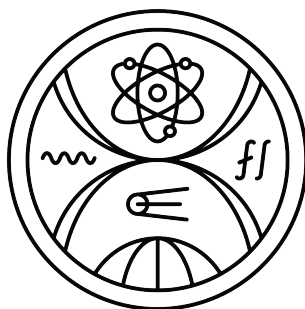
UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY



AKTÍVNA PERCEPCIA V ROBOTICKOM VIDENÍ

Diplomová práca

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY



AKTÍVNA PERCEPCIA V ROBOTICKOM VIDENÍ

Diplomová práca

Študijný program: Aplikovaná informatika
Študijný odbor: Aplikovaná informatika
Školiace pracovisko: Katedra aplikovanej informatiky
Vedúci práce: prof. Ing. Igor Farkaš, Dr.



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Patrik Modrovský
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Aktívna percepcia v robotickom videní
Active perception in robotic vision

Anotácia: Proces percepcie v robotickom kontexte je charakteristický aktívnou interakciou s objektmi počas učenia. Tieto procesy súčasne umožňujú "lacné" generovanie rozmanitých tréningových dát, ktoré sa dajú využiť na rôzne úlohy, napríklad robustne rozpoznávať objekty.

Cieľ: 1. Štúdium literatúry v oblasti aktívnej percepcie.
2. Návrh modelu neurónovej siete ktorá umožní robotovi skúmať objekt držaný v ruke v rôznych uhloch.
3. Vyhodnotenie funkčnosti a úspešnosti modelu a návrh možných vylepšení.

Literatúra: End to end active perception, I. Kostrikov, D. Erhan & S. Levine, NIPS 2016, http://www.dumitru.ca/files/DL_symposium_active_perception.pdf
Active Perception and Representation for Robotic Manipulation, Y.Zaky, G. Paruthi, B. Tripp, J. Bergstra, <https://arxiv.org/abs/2003.06734>

Vedúci: prof. Ing. Igor Farkaš, Dr.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 16.10.2022

Dátum schválenia: 06.11.2022
prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

Bratislava, 2023

.....
Bc. Patrik Modrovský

Acknowledgement

do later

Abstract

later

Keywords: robots

Abstrakt

later

Klíčové slová: roboti

Contents

1	Introduction	2
1.1	Neural networks	2
1.1.1	History	2
1.1.2	Different models of neural networks	3
1.1.3	Creating neural networks	5
1.2	Reinforcement learning	9
1.2.1	Introduction to RL	9

Motivation

nic moc zatiaľ

Chapter 1

Introduction

1.1 Neural networks

An Artificial Neural Network (ANN) is a processing system made from large number of interconnected simple processing elements - neurons. Architecture ANNs was inspired by structure of biological brain - both brain and ANN process information similarly and learns from examples.

This creates different approach to problem solving compared to conventional computing. Instead of predictable algorithmic approach, ANNs allows solving problems without understanding of problem, but with lower accuracy. [6]

1.1.1 History

In 1943 neurophysiologist Warren McCulloch and mathematician, Walter Pitts developed first model of ANN. Neurons in their models had fixed threshold and were able to simulate simple logic functions. [6]

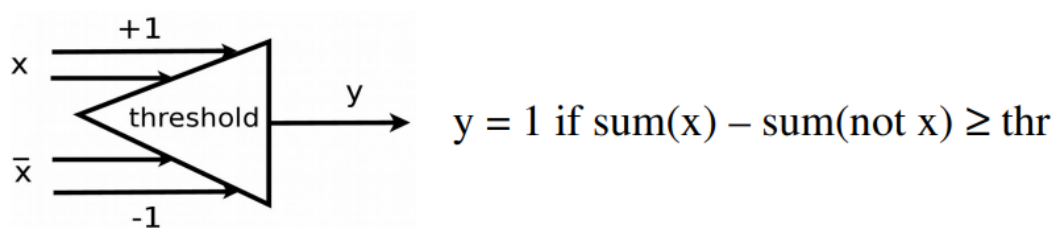


Figure 1.1: Model of an artificial neuron according to McCulloch and Pitts [4]

After initial period of enthusiasm and few other contributions to the field, period of frustration followed. During that time there were only few researchers continued work on solving problems with neural networks. During this time Klopff developed basis for learning and Werbos developed back-propagation algorithm.

This period lasted up to the 1980s, when multiple new contributions renewed wider

interest. Hopfield described recurrent neural networks and presented paper *Neural Networks and Physical Systems with Emergent Collective Computational Abilities*. [6]

Neural networks become recently much more publicly known, caused by advances in computing power allowing much bigger number of neurons. It allowed for much more complex tasks as image generation from natural language prompts or chatbots capable of deeper conversations.

1.1.2 Different models of neural networks

There are hundreds of different models and architectures of neural networks with different advantages and disadvantages for different purposes. Some of the most used architectures are:

- Single perceptrons
- Multi-layer perceptrons
- Convolutional neural network
- Recurrent neural network
- Self-Organizing Maps
- Radial Basis Function Network
- Hopfield autoassociative memory

In practise, multiple architectures are often combined to achieve desired outcome. [4]

Simple perceptron

It was proposed in 1958 by American psychologist, F. Rosenblatt. It is more general model, with free parameters, stochastic connectivity and threshold elements. [4]

Activation of this perceptron consists of weighted sum of inputs, from which we subtract threshold θ (1.1). Result is then run through activation function, either uni/bipolar for discrete perceptron or sigmoid for continuous perceptron. In practise, threshold is expressed as another weighted input x_{n+1} , with static value of -1, commonly called bias (1.2).

$$o_j = f\left(\sum_{i=1}^n w_{ij}x_i - \theta_j\right) \quad (1.1)$$

$$o_j = f\left(\sum_{i=1}^{n+1} w_{ij}x_i\right) \quad (1.2)$$

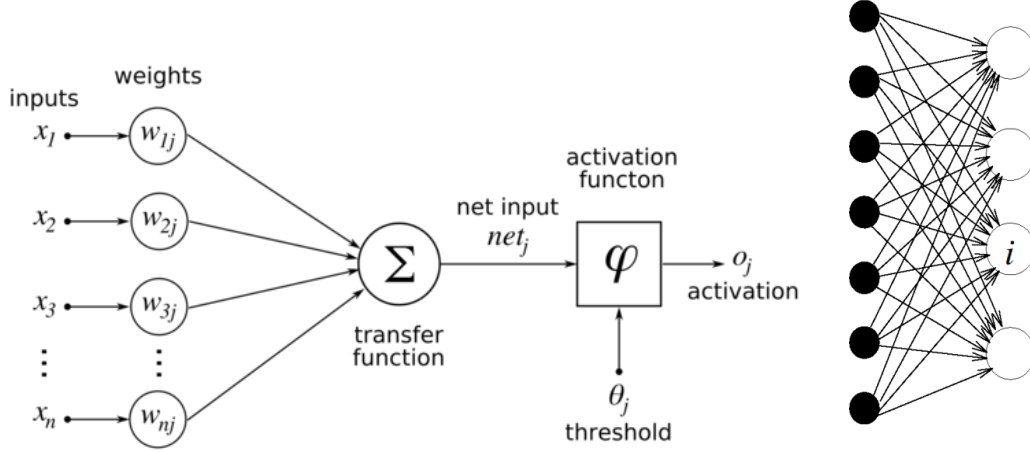


Figure 1.2: Model of simple perceptron and layer of simple perceptrons [4]

Learning rule for adjust weights in discrete perceptron:

$$w_j(t+1) = w_j(t) + \alpha(d - y)x_j \quad (1.3)$$

And for continuous perceptrons:

$$w_j(t+1) = w_j(t) + \alpha(d^{(p)} - y^{(p)})f'x_j \quad (1.4)$$

Where α (*Alpha*) represents learning rate.

Simple perceptron can solve only lineary separable classes. This caused loss of interest in neural networks in many researchers 1970s as many real problems are lineary non-separable, thus simple perceptron cannot be used to solve them.

Multi-layer perceptron

Multi-layer perceptron(MLP) is probably one of most used architectures today. It is generalisation of simple perceptrons and it contains additional hidden layers. MLP originates from *Rumelhart & McClelland: Parallel distributed processing* but was earlier described by Werbos in 1974. It is response to critique on perceptrons.[4]

In MLP Activations are similar to activations in simple perceptron. Their distinction comes from addition of hidden layers and additional weights. Due this change, output layer is calculated from hidden layer, instead of inputs.

$$h_k = f\left(\sum_{j=1}^{n+1} v_{kj}x_j\right) \quad (1.5)$$

$$y_i = f\left(\sum_{k=1}^{q+1} w_{ik}h_k\right) \quad (1.6)$$

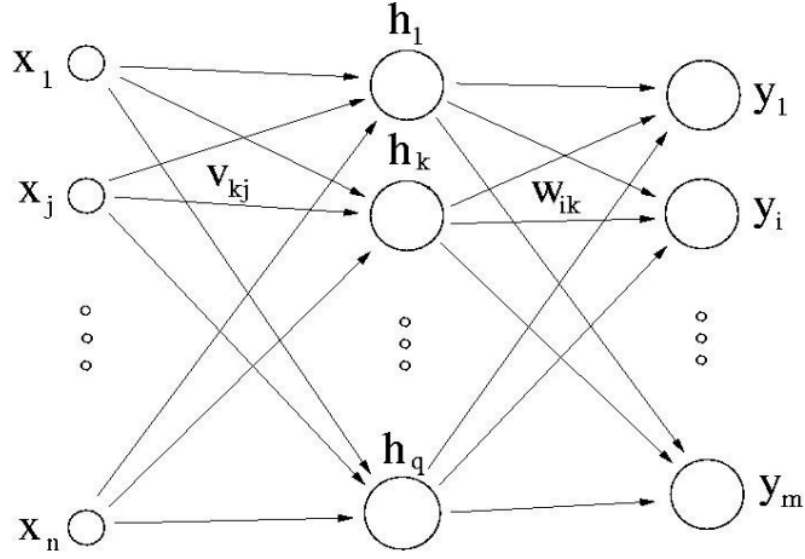


Figure 1.3: Model of multi-layer neural network [4]

Activation functions on hidden layers doesn't have similar constraints as output function, but linear function should not be used, as it doesn't add any depth to network. Common activation functions are sigmoid or hyperbolic tangent function.

Learning of MLP uses back-propagation algorithm with different equations for hidden and output layers. Weight adjustments are calculated backwards from output layers to the first hidden layer.[4]

Equation for calculating Hidden-output weights:

$$w_{ik}(t+1) = w_{ik}(t) + \alpha \delta_i h_k \text{ where } \delta_i = (d_i - y_i) f'_i \quad (1.7)$$

And for Input-hidden weights:

$$v_{kj}(t+1) = v_{kj}(t) + \alpha \delta_k x_j \text{ where } \delta_k = \left(\sum_i w_{ik} \delta_i \right) f'_k \quad (1.8)$$

1.1.3 Creating neural networks

To successfully create a neural network for accurate classification or prediction tasks, three key components play major role: high-quality dataset, the selection of an appropriate neural network model, and the identification of optimal hyperparameters for the training process itself.

Training process

During training, model iterates through dataset, predict outcome and calculates weight changes based on error. One such iteration is called **epoch**.

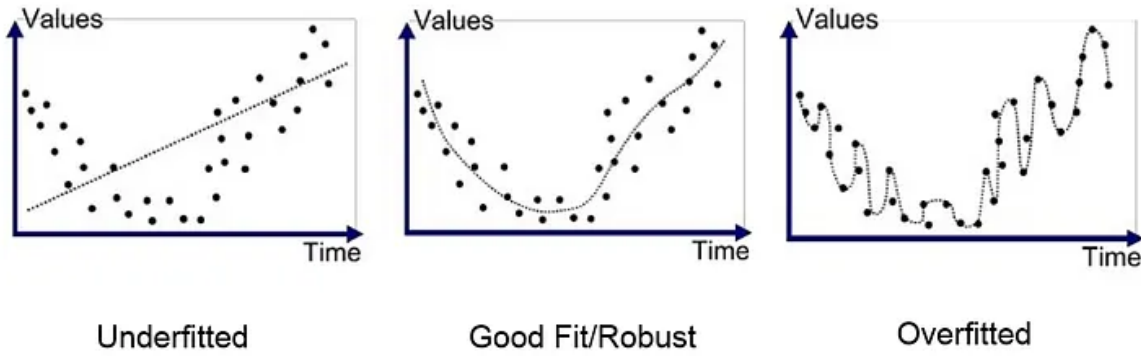


Figure 1.4: Underfit, Good fit and Overfit [2]

One training iteration usually consist of these steps:

- for each input x and desired output d (in random order)
 1. compute output: $y = ?(Wx)$
 2. compute error: $e = d - y$
 3. compute adjustment: $\Delta W = ?(W, x, y, e)$
 4. adjust weights: $\Delta W(t + 1) = W(t) + \alpha \Delta W$

This iteration continues until certain stopping criteria is meet, usually number of passed epochs or accuracy of model. Properly setting stopping criteria is important in order to prevent overfitting or underfitting of model for certain data. **Underfitting** is when model didn't properly capture desired problem. **Overfitting** is when model is trained very well for training data, but reacts too much on random outliers or noise, making it very inaccurate in generalisation (Fig. 1.5). [2]

In relation of correct fitting and training length, despite training error decreasing, at certain point validation error stops decreasing and start increasing, as model is becoming more overfit, reacting to noise in the dataset.

This is also called *sequential training*. Other used type of training is called *batch training*. In batch, instead of iterating through input-output pairs is everything calculated in parallel. This is done through matrix multiplication and can significantly speed up calculations, at cost of higher memory requirement and harder implementation.[4]

Datasets

Selection of proper dataset is crucial for training process - dataset should properly represent data from problem. In practise, getting ideal datasets is complicated and often impossible and contain too small sample of data. This can be partially corrected by using data augmentation.

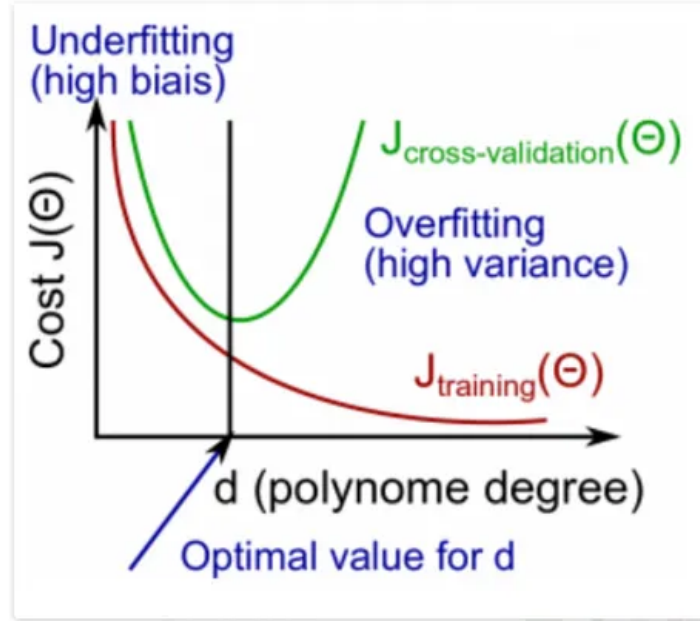


Figure 1.5: Error in relation to training length [2]

Data augmentations refers to changing available dataset to better suit task needs or generating new data based on present one. As example, in article *Improving handgun detection through a combination of visual features and body pose-based data* [8], authors edited original dataset of high quality images by editing lighting and "zooming out" to make humans on them appear smaller 1.6. This changes simulated of CCTVs, that often cover poorly lit big areas. They also used generation of new data by flattening 3D pose data to 2D from multiple points of view 1.7.

Although we could use dataset as is, and training error could be sufficiently minimised. Model wouldn't be good enough at generalisation. This is why datasets are split into two or three parts:

- Training set
- Validation set
- Testing set

Training set is used for training itself - it is used every epoch and it directly affects model weights. **Validation set** is used for evaluation of model and fine tune its hyperparameters. **Testing set** is used for measurement of model performance after training was finished (Fig. 1.8).

There is no hard rule on how dataset should be split as it depends on model and task, but in general, 80% – 10% – 10% for training, validation and testing is good split to start from.[1]

Simplest way to distribute data between splits is **Random sampling**. In random sampling data are picked at random from full dataset. This work best for balanced



Figure 1.6: Unedited and edited images [8]

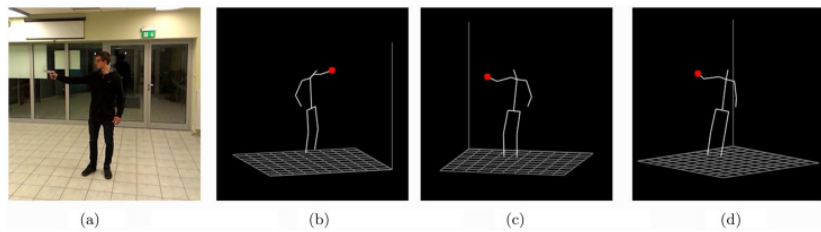


Figure 1.7: Different points of view [8]

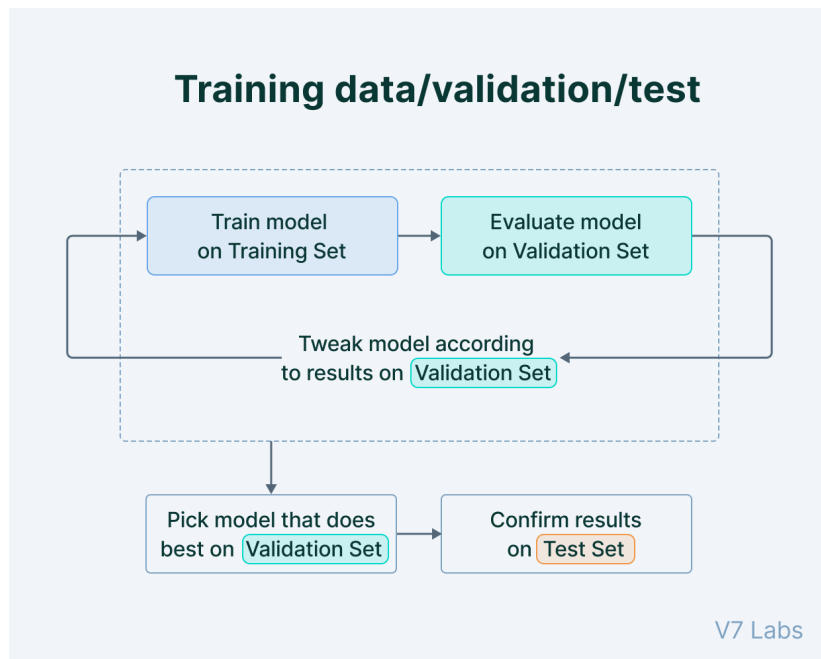


Figure 1.8: Difference between training, validation and testing sets [1]

datasets, but in unbalanced ones it can cause bias towards one category. For example in dataset with 800 images of dogs and 200 of cats, training set could get most of dog images, making cat detection unreliable. For these tasks **Stratified sampling** is used. In stratified sampling, data is picked from each category separately in same ratio as split between training, validation and testing sets.[1]

1.2 Reinforcement learning

In the realm of artificial intelligence, learning methods can be categorized into three types: supervised learning, unsupervised learning, and reinforcement learning, each having its own uses and advantages.

Supervised learning learns from a labelled dataset, where each input has a corresponding desired output. The goal is generalize and make accurate predictions for new, unseen data. This method is commonly used for regression and classification tasks, where the algorithm iteratively adjusts its parameters to minimize the difference between predicted and true values. Accuracy of predictions is heavily dependent of quality of dataset.

Unsupervised learning learns on unlabelled datasets. It discovers inherent patterns or structures within the data. Common tasks include clustering similar data points or reducing dataset dimensionality. Unsupervised learning is used for exploring large datasets, uncovering hidden relationships, and helping in data visualization.

Reinforcement learning learns from interaction with environment through actions and rewards. Its goal is mapping environment states to actions order to maximize long term reward. It is used in robotics, self-driving cars or playing games. Two main characteristics of RL are trial and error and delayed reward.

1.2.1 Introduction to RL

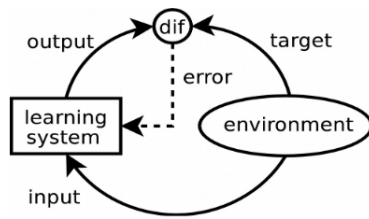
Reinforcement learning is mapping actions for every state in order to maximize rewards. Rewards are not know to the learner and have to be discovered by trying them. In more complex cases, actions affects all subsequent states and rewards.

Reinforcement learning simultaneously refers to problem, class of solution methods and field that studies this problem.

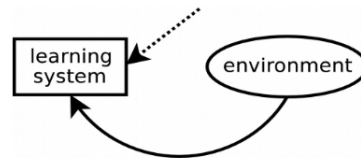
In RL problem, a learning agent must be able to sense state of the environment, take actions that can affect said state and have goal or goals relating to the state of environment.

Reinforcement learning is different from supervised learning. Supervised learning learns from already mapped set of data, where every label has a correct action and model tries to generalise for previously unseen data, for example correctly categorize

supervised (with teacher)



unsupervised (self-organized)



reinforcement learning (partial feedback)

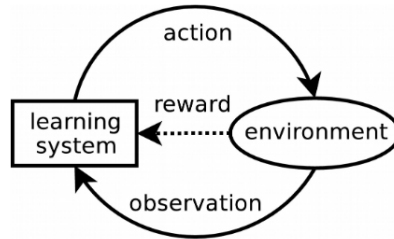


Figure 1.9: Different categories of AI learning methods [4]

animal in picture or calculate y from input x . Despite its usefulness, it is inadequate for learning from interactions, as it is often impractical to obtain representative examples of desired action for all situations in which the agent has to act. Agent must be able to learn from its own experience in order to find best action for every state.[9]

Bibliography

- [1] Pragati Baheti. Train test validation split: How to & best practices [2023]. 2023.
- [2] Anup Bhande. What is underfitting and overfitting in machine learning and how to deal with it. 2018.
- [3] Arden Dertat. Applied deep learning - part 4: Convolutional neural networks. 2017.
- [4] Igor Farkaš. Neural networks. page 192. Knížničné a edičné centrum FMFI UK, 2011.
- [5] Dominique A. Heger. An introduction to artificial neural networks (ann)-methods , abstraction , and usage. 2015.
- [6] Bohdan Macukow. Neural networks – state of art, brief history, basic models and architecture. In Khalid Saeed and Władysław Homenda, editors, *Computer Information Systems and Industrial Management*, pages 3–14, Cham, 2016. Springer International Publishing.
- [7] Hai Nguyen and Hung La. Review of deep reinforcement learning for robot manipulation. In *2019 Third IEEE International Conference on Robotic Computing (IRC)*, pages 590–595, 2019.
- [8] Jesus Ruiz-Santaquiteria, Alberto Velasco-Mata, Noelia Vallez, Oscar Deniz, and Gloria Bueno. Improving handgun detection through a combination of visual features and body pose-based data. *Pattern Recognition*, 136:109252, 2023.
- [9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.
- [10] Youssef Zaky, Gaurav Paruthi, Bryan Tripp, and James Bergstra. Active perception and representation for robotic manipulation. *CoRR*, abs/2003.06734, 2020.